

Compte Rendu — TP4.5

API RESTful avec Rate Limiting et CORS

Matière	SoA et Microservices
Enseignant	Dr. Salah Gontara
Classe	4Info
Année universitaire	2025/2026
Prérequis	TP4 — API RESTful avec Express.js et SQLite3
Technologies	Node.js, Express.js, cors, express-rate-limit

Introduction

Ce TP s'appuie sur l'API RESTful développée lors du TP4 (Express.js + SQLite3 + routes CRUD sur /personnes). L'objectif est d'améliorer la robustesse et la sécurité de l'API en ajoutant deux mécanismes essentiels : la gestion des CORS (Cross-Origin Resource Sharing) pour autoriser les requêtes multi-origines, et le Rate Limiting pour protéger l'API contre les abus et les attaques par déni de service.

Notions Théoriques

CORS — Cross-Origin Resource Sharing

Le CORS est un mécanisme de sécurité implémenté par les navigateurs web. Par défaut, un navigateur bloque toute requête JavaScript effectuée vers un domaine différent de celui de la page courante (politique Same-Origin). Le CORS permet de lever cette restriction de manière contrôlée en ajoutant des headers HTTP spécifiques dans la réponse du serveur.

Exemple concret : si un frontend Angular tourne sur `http://localhost:4200` et tente d'appeler une API Express sur `http://localhost:3000`, le navigateur bloque la requête sans configuration CORS. En activant CORS sur le serveur, on autorise explicitement cette communication.

Header HTTP	Valeur	Signification
Access-Control-Allow-Origin	*	Autorise toutes les origines
Access-Control-Allow-Origin	<code>http://localhost:4200</code>	Autorise une origine spécifique
Access-Control-Allow-Methods	GET, POST, PUT, DELETE	Méthodes HTTP autorisées

Rate Limiting — Limitation du débit

Le Rate Limiting est un mécanisme de protection qui limite le nombre de requêtes qu'une adresse IP peut effectuer dans un intervalle de temps donné. Il protège l'API contre les attaques DDoS (Distributed Denial of Service), les bots qui spamment l'API, et les abus d'utilisation.

Paramètre	Valeur dans le TP	Signification
windowMs	<code>15 * 60 * 1000</code>	Fenêtre de 15 minutes
max	100	Max 100 requêtes par IP par fenêtre
message	Texte d'erreur	Message retourné quand limite atteinte
Code HTTP retourné	429	Too Many Requests

Étape 1 : Installation des Modules

Commande exécutée

Terminal — Installation

```
npm install cors express-rate-limit
```

Capture — Installation

[Capture : Résultat npm install cors express-rate-limit]

Analyse

Le package "cors" est un middleware Express qui ajoute automatiquement les headers HTTP nécessaires pour autoriser les requêtes cross-origin. Le package "express-rate-limit" implémente un compteur de requêtes par IP avec une fenêtre temporelle glissante.

Vérification dans package.json

[Capture : package.json après installation — section dependencies]

Conclusion

Les deux packages sont installés et ajoutés automatiquement dans package.json. Ils seront ensuite importés et configurés comme middlewares dans index.js.

Étape 2 : Mise à Jour du Fichier index.js

Modifications apportées

Trois modifications ont été apportées au fichier index.js existant : ajout des imports cors et express-rate-limit, configuration du middleware CORS, et configuration du middleware Rate Limiting.

1. Import des nouveaux packages

index.js — Imports ajoutés

```
const cors = require('cors');  
const rateLimit = require('express-rate-limit');
```

2. Configuration CORS

index.js — Middleware CORS

```
// Autorise toutes les origines
```

```
app.use(cors());

// Pour restreindre à certains domaines :
// app.use(cors({ origin: ['http://localhost:3000',
// 'http://localhost:4200'] }));
```

`app.use(cors())` autorise toutes les origines. En production, il est recommandé de restreindre les origines autorisées en passant un tableau de domaines spécifiques pour éviter tout accès non autorisé.

3. Configuration Rate Limiting

index.js — Middleware Rate Limiting

```
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100,                  // 100 requêtes max par IP
  message: 'Trop de requêtes effectuées depuis cette IP, '
    + ' veuillez réessayer après 15 minutes.'
});
app.use(limiter);
```

Le paramètre `windowMs` définit la fenêtre de temps en millisecondes. Le paramètre `max` fixe le nombre maximum de requêtes autorisées par IP dans cette fenêtre. Après 100 requêtes, toute nouvelle requête reçoit un statut 429 Too Many Requests.

Capture — Fichier index.js mis à jour

[Capture : Début de index.js avec CORS et Rate Limiting configurés]

Ordre des middlewares

L'ordre d'application des middlewares dans Express est crucial. Les middlewares s'exécutent dans l'ordre où ils sont déclarés avec `app.use()`. CORS et Rate Limiting doivent être placés avant la définition des routes pour s'appliquer à toutes les requêtes entrantes.

Ordre	Middleware	Rôle
1	<code>express.json()</code>	Parse le corps JSON des requêtes
2	<code>cors()</code>	Ajoute les headers CORS à chaque réponse
3	<code>rateLimit()</code>	Compte et limite les requêtes par IP
4	Routes (GET, POST...)	Traitement de la requête

Conclusion

La mise à jour de `index.js` est minimale et non destructive : les routes CRUD existantes du TP4 sont conservées intactes. CORS et Rate Limiting sont appliqués globalement à toutes les routes grâce à `app.use()`.

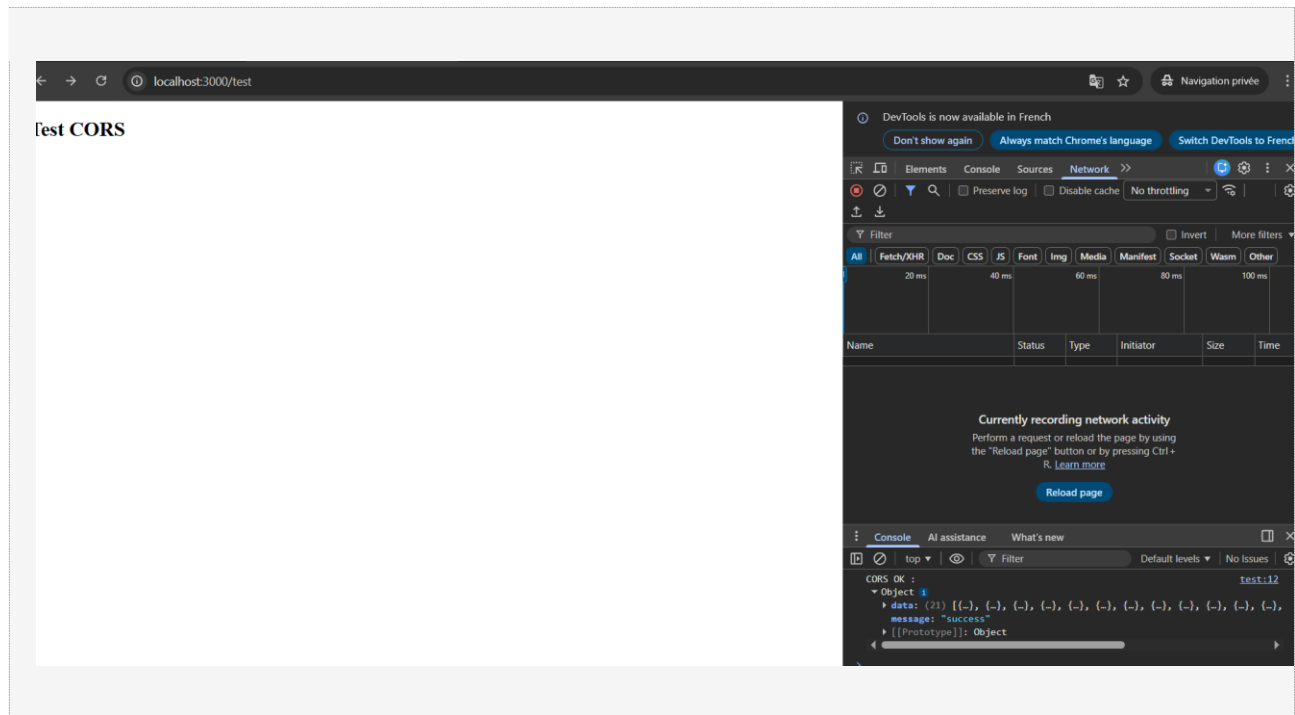
Étape 3 : Tests de l'Implémentation

Test 1 — Test CORS avec fichier HTML

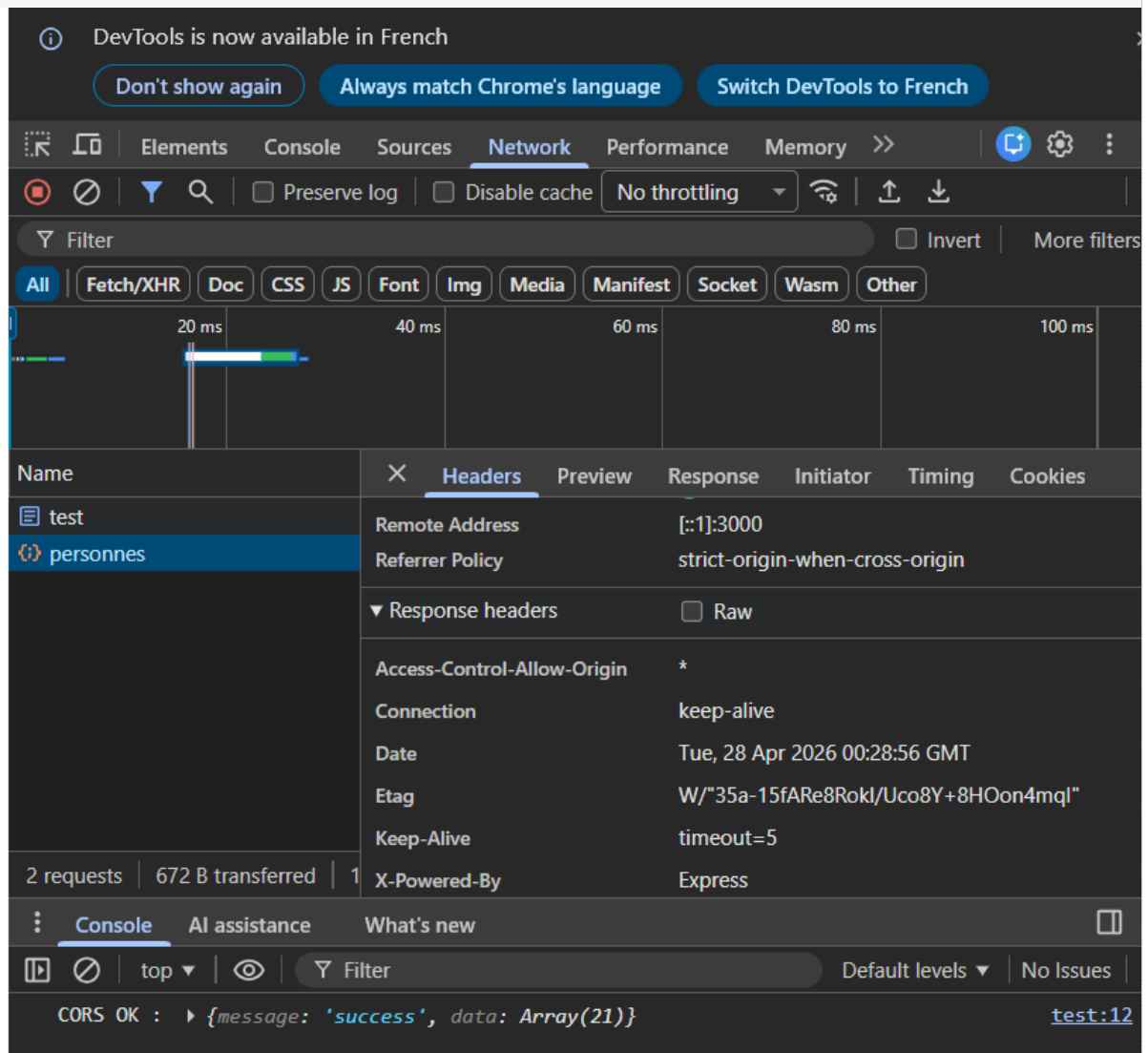
Procédure

Un fichier HTML de test (test-cors.html) a été créé. Il utilise l'API Fetch JavaScript pour effectuer une requête GET vers `http://localhost:3000/personnes` depuis le navigateur. Si CORS est correctement configuré, les données sont reçues sans erreur.

Capture — Console navigateur (F12) — CORS OK



Capture — Headers de réponse — Access-Control-Allow-Origin



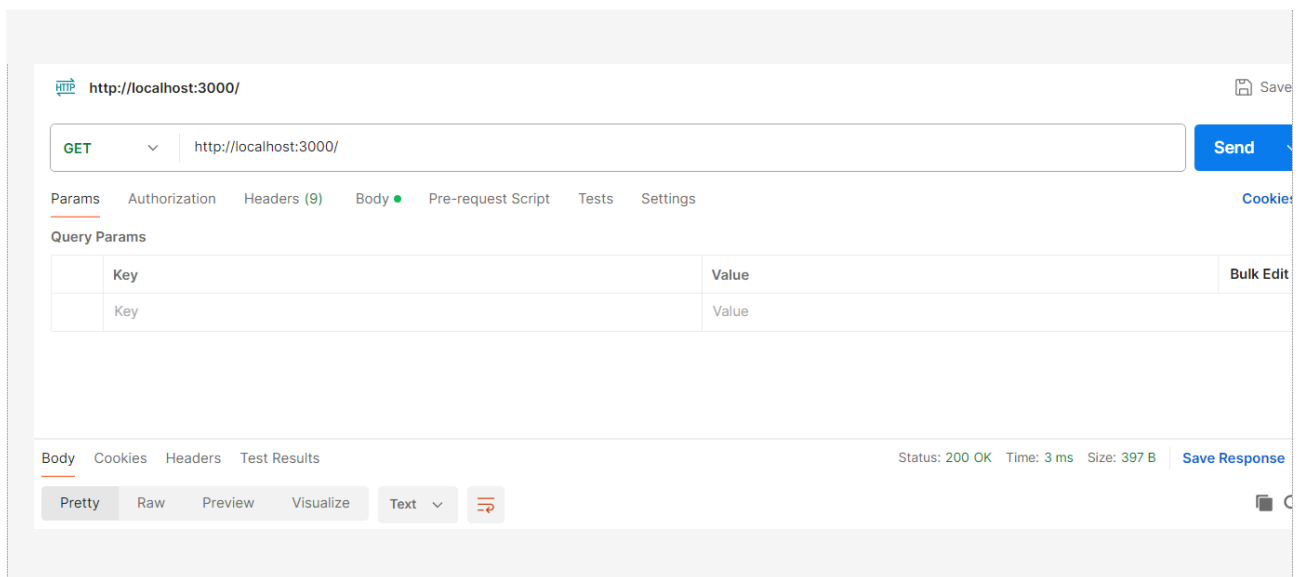
Le header "Access-Control-Allow-Origin: *" dans la réponse confirme que le serveur autorise les requêtes depuis n'importe quelle origine. Sans ce header, le navigateur aurait bloqué la réponse avec une erreur CORS.

Test 2 — Test Rate Limiting avec Postman

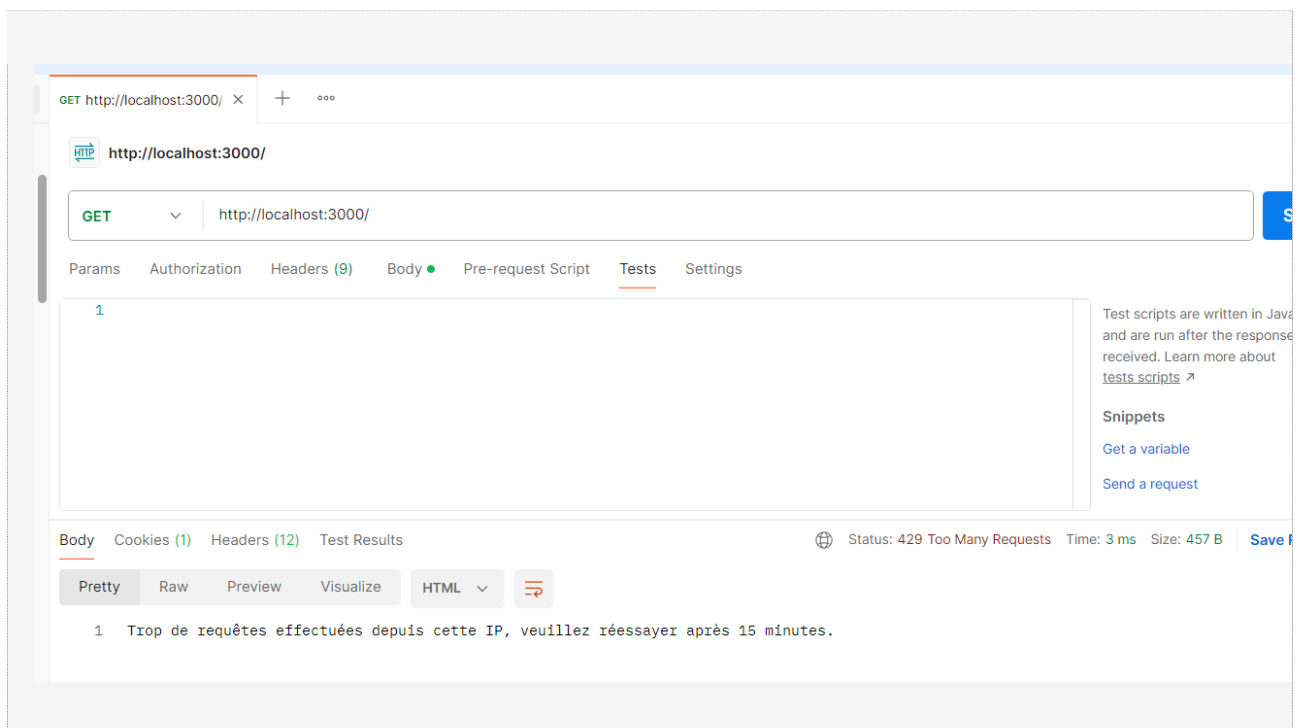
Procédure

La fonctionnalité Collection Runner de Postman a été utilisée pour envoyer plus de 100 requêtes GET vers `/personnes` en moins de 15 minutes depuis la même adresse IP. Les premières 100 requêtes reçoivent une réponse normale (200 OK). À partir de la 101ème requête, l'API retourne une erreur 429.

Capture — Requête normale (avant la limite)



Capture — Requête bloquée (après 100 requêtes)



Le message retourné est : "Trop de requêtes effectuées depuis cette IP, veuillez réessayer après 15 minutes." Ce comportement protège l'API contre les abus tout en informant clairement le client de la raison du refus.

Headers de Rate Limiting observés

Header	Valeur	Signification
X-RateLimit-Limit	100	Nombre max de requêtes autorisées
X-RateLimit-Remaining	N (décroît)	Requêtes restantes dans la fenêtre
X-RateLimit-Reset	Timestamp	Heure de réinitialisation du compteur

Conclusion

Les deux tests valident le bon fonctionnement des mécanismes implémentés. CORS permet aux applications frontend de communiquer avec l'API sans être bloquées par le navigateur. Le Rate Limiting protège efficacement le serveur en retournant un statut 429 après dépassement de la limite.

Conclusion Générale

Ce TP a permis d'enrichir l'API RESTful existante avec deux mécanismes de sécurité et de robustesse essentiels en production :

Mécanisme	Package	Bénéfice
CORS	cors	Autorise les requêtes cross-origin des frontends
Rate Limiting	express-rate-limit	Protège contre les abus et les attaques DDoS

Les deux mécanismes sont implémentés comme des middlewares Express, ce qui les rend simples à configurer et à maintenir. Leur placement avant les routes garantit qu'ils s'appliquent à toutes les requêtes entrantes. En production, il est recommandé de restreindre les origines CORS autorisées à une liste précise de domaines, et d'adapter les paramètres de Rate Limiting selon le trafic attendu.